

Caching – Why it's needed (News Feed context)

A **News Feed system** is:

- **Read-heavy** (scrolling feeds)
- **Latency-sensitive**
- Needs **scalability**

Without cache:

👉 Every scroll → DB hit → slow & expensive

With cache:

👉 Feed data served from **Redis/Memcached** in **milliseconds**

2 Caching Strategies (Core Concepts)

◆ 1. Read-Through Cache

Flow

Client → Cache → (Miss) → DB → Cache → Client

How it works

- App always reads from cache
- Cache itself fetches data from DB on miss

Example (News Feed)

- User opens app
- Cache miss for `feed:user:123`
- Cache loads feed from DB
- Stores it
- Returns to user

Pros

- ✓ Simple logic
- ✓ Good for read-heavy systems

Cons

- ✗ Cache miss penalty
- ✗ Cold start problem

Used when

- 👉 Feeds, timelines, profile data
-

♦ 2. Write-Through Cache

Flow

Client → Cache → DB

How it works

- Writes go to cache first
- Cache synchronously writes to DB

Example

- User likes a post
- Like count updated in cache
- Same update written to DB immediately

Pros

- ✓ Strong consistency
- ✓ Cache always fresh

Cons

- ✗ Higher write latency
- ✗ DB write on every update

Used when

- 👉 Counters, user settings
-

♦ 3. Write-Back (Write-Behind) Cache

Flow

Client → Cache → (Async) → DB

How it works

- Writes go only to cache
- DB updated later in batch

Example

- User likes multiple posts rapidly
- Cache updates instantly
- DB updated every few seconds

Pros

- ✓ Very fast writes
- ✓ Reduced DB load

Cons

- ✗ Data loss risk (cache crash)
- ✗ Eventual consistency

Used when

- 👉 Likes, views, impressions
-

3 Cache Invalidation (⚠️ Hardest Problem)

Why it's hard

- Data changes frequently
- Multiple users see same feed
- Cache might serve stale data

Common strategies

◆ TTL (Time to Live)

feed:user:123 → expires in 30 seconds

- ✓ Simple
 - ✗ Stale data till expiry
-

◆ **Explicit Invalidation**

New post created → delete feed caches

- ✓ Fresh data
 - ✗ Complex logic
-

◆ **Versioned Keys (Best Practice)**

feed:user:123:v45

- Increment version on update
- Old cache auto-ignored

- ✓ Clean & scalable
-

4 Redis vs Memcached (Interview Table)

Feature	Redis	Memcached
Data Types	Rich (List, Set, ZSet)	Key-Value only
Persistence	Yes	No
Pub/Sub	Yes	No
Use Case	Feeds, rankings	Simple caching
Eviction	Advanced	LRU

👉 News Feed → Redis preferred

5 News Feed Caching Architecture (Real-World)

Client



API Server



Redis (feed:userId)



Database

Optimizations

- Precompute feeds (fan-out on write)
- Use Redis **Sorted Sets** for ranking
- Use TTL + versioning



News Feed System – Full System Design

1 Requirements Clarification (Always start here)

Functional Requirements

- Users can:
 - Create posts (text/image/video)
 - Follow / unfollow users
 - View a personalized news feed (latest + relevant posts)
- Feed ordered by:
 - Time (primary)
 - Ranking signals (likes, relevance – optional)

Non-Functional Requirements

- **Low latency** (<200 ms)
- **High availability**
- **Scalable to millions of users**

- **Eventual consistency acceptable**
 - **Read-heavy system**
-

2 High-Level Architecture

Client



API Gateway



Feed Service



Redis Cache ↔ Feed Store (DB)



Post Service



Post DB

Additional systems:

- **Follow Service**
 - **Fanout Workers**
 - **Ranking Service**
 - **Message Queue (Kafka)**
 - **CDN (media)**
-

3 Core Data Models

User

```
{  
  "userId": 123,  
  "name": "Ankit"  
}
```

Post

```
{
  "postId": 987,
  "authorId": 456,
  "content": "...",
  "timestamp": 1700000000
}
```

Follow Graph

followerId → followeeId

4 Feed Generation Approaches (MOST IMPORTANT)

🔥 Option 1: Fan-Out on Write (Push Model) – Preferred

Flow

User posts



Fetch followers



Push postId into each follower's feed

Storage

- Redis **Sorted Set**

feed:user:123 → (postId, timestamp)

Pros

- ✓ Fast feed read
- ✓ Low read latency

Cons

- ✗ Expensive for users with many followers
-

🔥 Option 2: Fan-Out on Read (Pull Model)

Flow

User opens feed

↓
Fetch followees
↓
Fetch their posts
↓
Merge & sort

Pros

✓ Cheap writes

Cons

✗ Very slow reads
✗ Heavy DB joins

🏆 Hybrid Approach (Industry Standard)

- Fan-out on write for normal users
 - Fan-out on read for celebrities
-

5 Feed Storage Design (Redis)

Redis Sorted Set

```
ZADD feed:user:123 timestamp postId  
ZREVRANGE feed:user:123 0 50
```

Why Sorted Set?

- Automatic ordering
 - Fast pagination
 - Score = timestamp or rank score
-

6 Feed Read Flow (Read-Through Cache)

Client → Feed API

↓
Check Redis (feed:userId)
↓
Cache Hit → return feed
↓
Cache Miss → fetch from DB → store in Redis → return

Cache Key Design

feed:user:{userId}:v{version}

7 Feed Write Flow (Fan-Out on Write)

User creates post
↓
Post Service saves post
↓
Publish event to Kafka
↓
Fanout Workers
↓
Update Redis feeds of followers

Kafka Topic

post_created

8 Ranking & Ordering

Basic Ranking

score = timestamp

Advanced Ranking (optional)

score = (likes * weight1) + (comments * weight2) + recency

Separate Ranking Service

- Updates score asynchronously

- Reorders Redis sorted sets
-

9 Cache Invalidation (Hardest Part)

Problems

- New post
 - Deleted post
 - Follow/unfollow
 - Likes/comments changing rank
-

Solutions

♦ TTL

`feed:user:123 → TTL 30 sec`

♦ Explicit Invalidation

`User unfollows → delete feed cache`

♦ Versioned Keys (Best)

`feed:user:123:v42`

Increment version on:

- Follow/unfollow
 - Major feed change
-

10 Handling Celebrities (Hot Users)

Problem:

- 10M followers → cannot fan-out

Solution:

- Store celebrity posts separately
- Merge at read time

```
feed = userFeed + celebrityPosts
```

11 Pagination Strategy

Cursor-based Pagination (Preferred)

```
GET /feed?cursor=timestamp
```

Avoid:

✗ Offset pagination (slow for large data)

12 Database Choices

Data	DB
Posts	MySQL / Cassandra
Feed	Redis
Followers	Graph DB / Cassandra
Events	Kafka

13 Failure Handling

Redis Failure

- Fallback to DB
- Rebuild feed

Kafka Failure

- Retry consumers
- Dead Letter Queue

Data Loss Prevention

- Persist posts first
 - Cache is rebuildable
-

14 Scaling Strategy

Horizontal Scaling

- Shard Redis by userId
- Partition Kafka by authorId

Read Scaling

- Redis replicas
 - CDN for media
-

15 Security & Abuse Prevention

- Rate limiting on post creation
 - Spam detection
 - Block/mute users
-

16 Final Architecture Diagram (Mental Model)

Client



API Gateway



Feed Service



Redis (Sorted Sets)



Kafka ← Fanout Workers



Post DB

Interview Summary (Say This)

“The news feed is a read-heavy system. We use **fan-out on write** with Redis sorted sets for fast reads, a hybrid model for celebrities, Kafka for async fanout, and versioned cache keys to handle invalidation. The system is eventually consistent and horizontally scalable.”